

Development Summary

1. Project Overview

The SpendWise Expense Tracker is a full-stack solution designed to provide users with a streamlined way to monitor their personal finances. We have built a secure, responsive web application that allows users to register accounts, log daily expenses, and visualize their spending habits through dynamic reports. The architecture consists of a high-performance FastAPI backend and a modern React frontend, ensuring a smooth and scalable user experience.

2. Implemented Services

Backend API

A RESTful service built with FastAPI that manages data persistence, business logic, and secure user authentication.

- `app/main.py`: The central entry point that configures the API, handles CORS, and initializes database tables.
- `app/core/`: Contains configuration settings, database connection logic, and security utilities for JWT handling.
- `app/models/models.py`: Defines the database structure for Users, Categories, and Expenses.
- `app/schemas/schemas.py`: Manages data validation for incoming requests and outgoing responses.
- `app/api/v1/`: Contains specific endpoint logic for user authentication, expense CRUD operations, and monthly report generation.

Frontend Web

A modern, interactive user interface built with React and Vite, styled using Tailwind CSS for a professional look and feel.

- `src/services/api.js`: A centralized Axios configuration that handles API requests and automatic JWT authentication headers.
- `src/App.jsx`: The primary application shell managing protected routing and navigation.
- `src/pages/`: Includes dedicated views for User Login, Account Registration, and the main Spending Dashboard.

- `src/components/`: Reusable UI components including a Pie Chart for spending breakdowns, an Expense Entry form, and a searchable Expense List.

3. Local Setup Instructions

To get the SpendWise application running on your local environment, please follow these steps:

Prerequisites:

- Python 3.9+ installed.
- Node.js 16+ installed.
- A running PostgreSQL instance.

Step-by-Step Setup:

1. Database Initialization: Create a new PostgreSQL database named `spendwise`.
2. Backend Configuration:
 - Navigate to the `/backend` folder.
 - Create a virtual environment: `python -m venv venv`.
 - Activate the environment and install dependencies: `pip install -r requirements.txt`.
 - Set your environment variables: `DATABASE_URL` (pointing to your PostgreSQL instance) and `JWT_SECRET` (a unique string for security).
 - Launch the API: `uvicorn app.main:app --reload`.
3. Frontend Configuration:
 - Navigate to the `/frontend` folder.
 - Install the necessary packages: `npm install`.
 - Create a `.env` file and set `VITE_API_URL=http://localhost:8000/api/v1`.
 - Start the web interface: `npm run dev`.

4. Development Notes

- Automated Data Seeding: To simplify the initial setup, the backend automatically populates the database with standard expense categories (Food, Transport, Rent, etc.) the first time it runs.
- Security Protocol: User sessions are managed via JSON Web Tokens (JWT). These are stored in the browser's local storage and are automatically cleared upon logout or token expiration.
- Data Precision: For this initial build, currency is handled using floating-point

numbers. For a production-grade financial release, we recommend migrating to specific numeric types to ensure absolute decimal precision.

- Reporting Logic: The dashboard summary is currently configured to aggregate data based on the current calendar month, providing users with an immediate "at-a-glance" view of their recent spending.
- CORS Policy: Cross-Origin Resource Sharing is currently set to allow all origins to facilitate ease of testing; this should be restricted to your specific domain during production deployment.